



INSTITUTO POLITECNICO NACIONAL

Escuela Superior de Computo

Arquitectura de las Computadoras

Medio y Full Sumador en Verilog

Profesor: Miguel Angel Aleman Arce

Alumno: Gutiérrez Espinosa Jordan Gabriel

Grupo: 5CV1

Ciclo: 2022-2023(1)

Medio sumador

El medio sumador binario usando verilog se puede resumir en uso de un XOR para saber el resultado ya que como se ve en la Figura 1, solo en el sumador se toma en cuenta 1 si son los 2 números que se usan diferentes, si los 2 son 0 se cuenta cómo 0 pero si los 2 son 1 la suma da 0 pero se muestra que hubo un Carrier, ósea otro numero de la suma para tomar en cuenta en ese caso se utiliza un and con esas 2 variables que se representara en ese carry. Ya en el código que se utiliza se muestra en la Figura 2, su representación en compuestas lógicas en la Figura 3 y su Modulo de estimulación y ejecución en la Figura 4.

Half Adder Truth Table			
A	B	Carrier	Sum
0	0	0	0
1	0	0	1
0	1	0	1
1	1	1	0

Figura 1, muestreo de la suma binaria

```
module mediosumador(  
    input a,  
    input b,  
    output sum,  
    output carry);  
  
    assign sum = a ^ b;  
    assign carry = a & b;  
  
endmodule
```

Figura 2, Código de Verilog sobre el Medio sumador

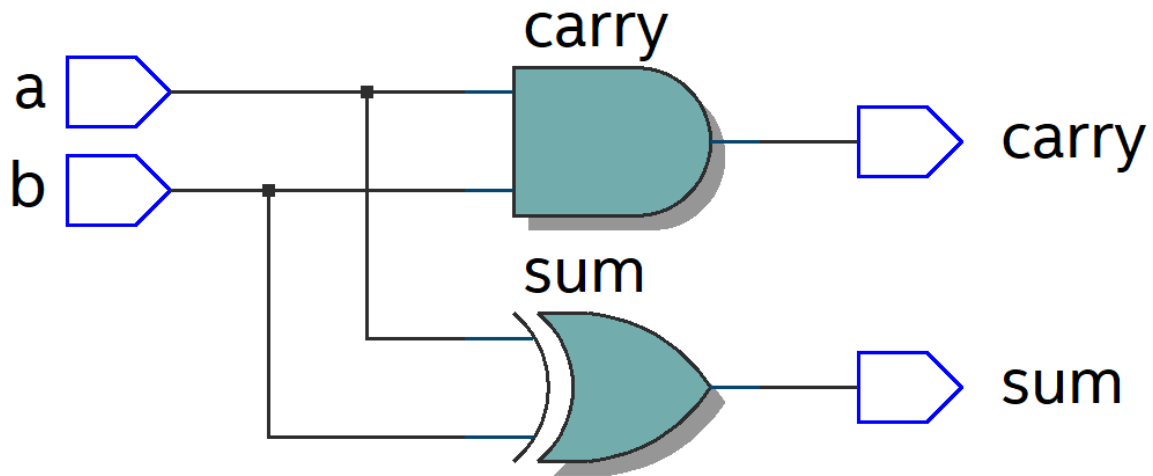


Figura 3, Representación de Compuestas Lógicas Medio Sumador

```

module mediosumador_tb;
  reg a;
  reg b;
  wire sum;
  wire carry;
  mediosumador uut (
    .a(a),
    .b(b),
    .sum(sum),
    .carry(carry)
  );

  initial begin
    a = 0;
    b = 0;
    #10;
    a = 0; b = 0;
    #10;
    a = 0; b = 1;
    #10;
    a = 1; b = 0;
    #10;
    a = 1; b = 1;
    #10;
    $finish;
  end
endmodule

```

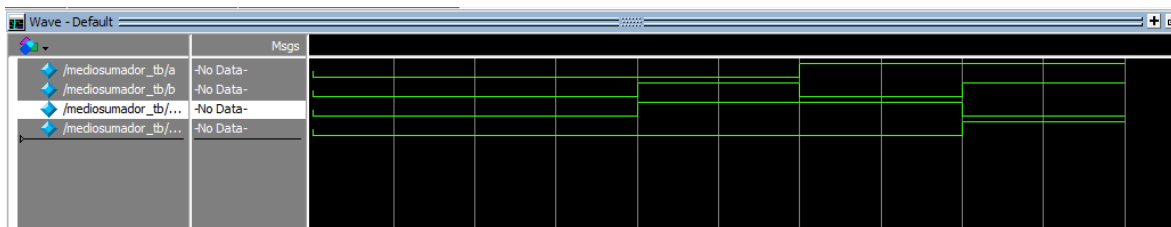


Figura 4, Simulación

Full sumador

El sumador completo como se muestra en la Figura 5, es el que toma en cuenta si existe un corredor que lo siga en el momento de la suma, por lo cual se utiliza 2 medios sumadores como se muestra en el código de la Figura 6, uno tomara en cuenta la suma entre los valores de entrada normales y el otro sumara el resultado de esa suma y lo sumara a otro sumador con el careador, ya al ultimo si cualquier de los 2 sumadores saca un careado se le introduce a un OR para que solo saquemos un resultado como se muestra en la Figura 7, ya solo se muestra la Simulación en la Figura 8.

Full Adder Truth Table				
A	B	Cin	Cout	Sum
0	0	0	0	0
1	0	0	0	1
0	1	0	0	1
1	1	0	1	0
0	0	1	0	1
1	0	1	1	0
0	1	1	1	0
1	1	1	1	1

Figura 5, Sumador Completo.

```
module SumadorCompleto
(
    input a,
    input b,
    input cin,
    output sum,
    output cout);

    wire ha1_sum, ha1_carry, ha2_sum, ha2_carry;
    mediosumador ha1(a, b, ha1_sum, ha1_carry);
    mediosumador ha2(ha1_sum, cin, ha2_sum, ha2_carry);
    assign sum = ha2_sum;
    assign cout = ha1_carry | ha2_carry;
endmodule

module SumadorCompleto_tb;
```

Figura 6, Código Sumador Completo.

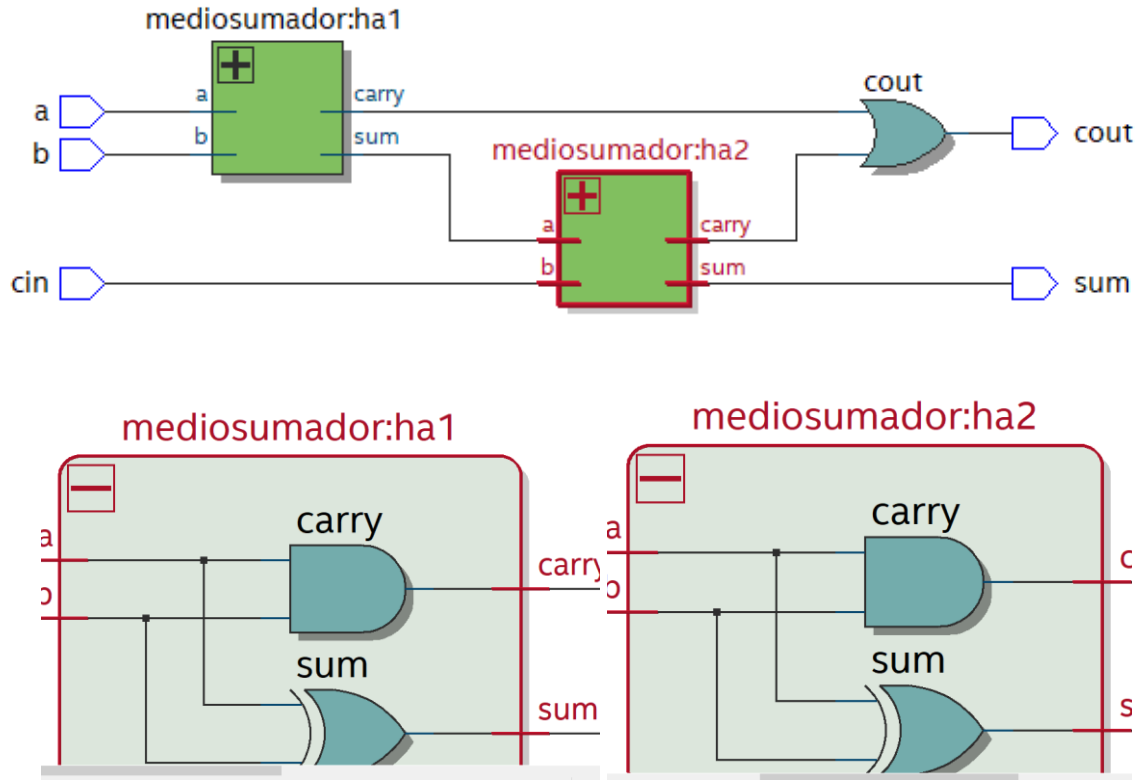


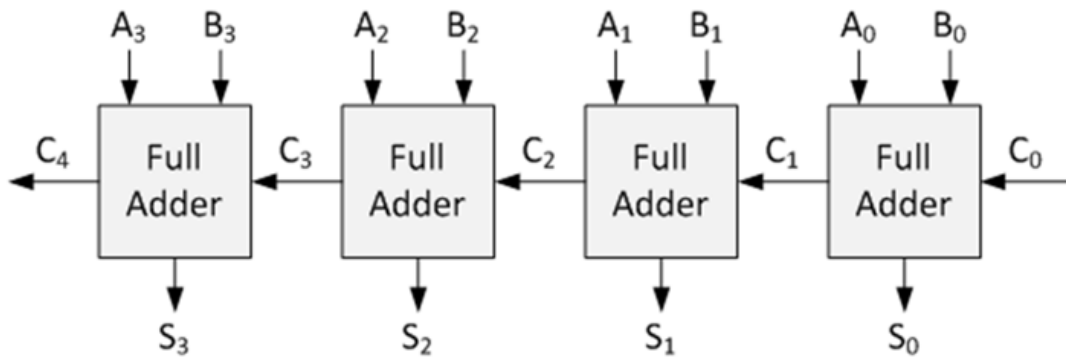
Figura 7, Compuertas Logicas del Sumador Completo.



Figura 8, Simulación del Sumador Completo.

Ripple Carry Adder

el sumador de 4 bits como se muestra en la Figura 9, es una combinación de 4 sumadores completos, como se muestra en el código de la Figura 10 el primero agarra los primeros bit de `a` y `b` sin acarreado y hace su suma, manda el resultado de su suma a la salida y manda el resultado de su acarreado al 2 el cual hará el mismo procedimiento como el tercero, ya el cuarto indica el resultado de la suma y si en su ejecución hay un acarreado, ya en la Figura 11 se muestra este sumador de 4 bits con puestas lógicas y el la Figura 12 se muestra su simulación.



Ripple Carry Adder (4-bit) Block Diagram

Figura 9, Sumador de 4 bit tipo triple acarreador.

```
module sumador4bits(
    input [3:0] a,
    input [3:0] b,
    output [3:0] sum,
    output Fcarry);

    wire [3:0] carry;

    Sumadorcompleto fa0(a[0], b[0], 1'b0, sum[0], carry[0]);
    Sumadorcompleto fa1(a[1], b[1], carry[0], sum[1], carry[1]);
    Sumadorcompleto fa2(a[2], b[2], carry[1], sum[2], carry[2]);
    Sumadorcompleto fa3(a[3], b[3], carry[2], sum[3], carry[3]);
    assign Fcarry=carry[3];
endmodule
```

Figura 10, Código Sumador de 4 bit tipo triple acarreador.

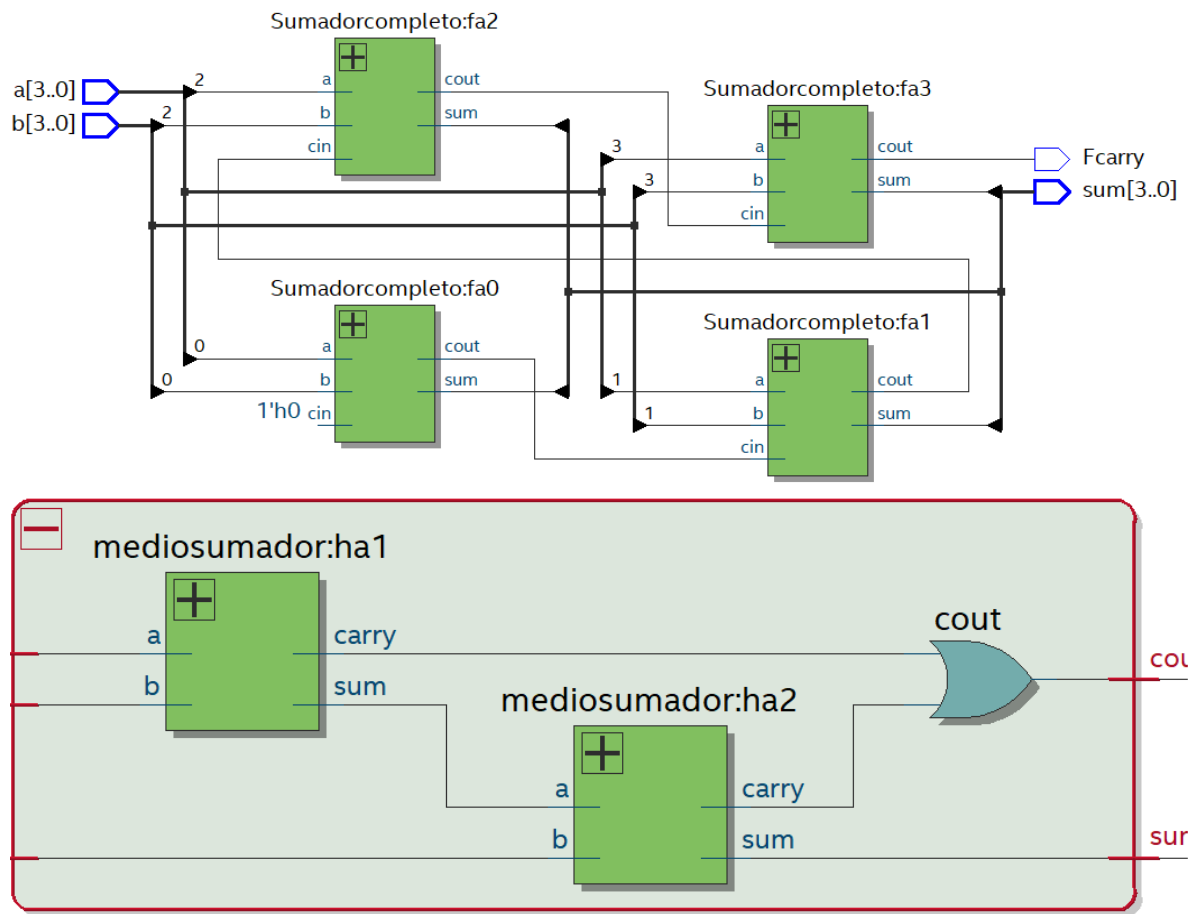


Figura 11, compuertas Lógicas del Sumador de 4 bit tipo triple acarreador.

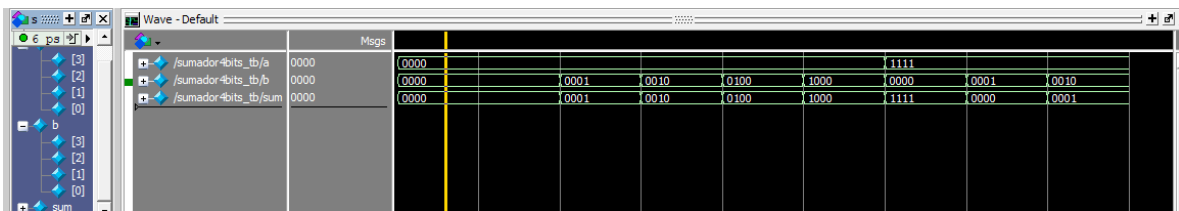


Figura 12, Simulación del Sumador de 4 bit tipo triple acarreador.

Sumador de 8 bits

Ya por ultimo en la implementación del de 8 se agrega 4 nuevos bloques como se muestra en la Figura 13 sobre el código y en la 14 sobre su representación en compuertas logicas, y se simula como se muestra en la Figura 14.

```

module sumador4bits(
    input [7:0] a,
    input [7:0] b,
    output [7:0] sum,
    output Fcarry);

    wire [7:0] carry;

    Sumadorcompleto fa0(a[0], b[0], 1'b0, sum[0], carry[0]);
    Sumadorcompleto fa1(a[1], b[1], carry[0], sum[1], carry[1]);
    Sumadorcompleto fa2(a[2], b[2], carry[1], sum[2], carry[2]);
    Sumadorcompleto fa3(a[3], b[3], carry[2], sum[3], carry[3]);
    Sumadorcompleto fa4(a[4], b[4], carry[3], sum[4], carry[4]);
    Sumadorcompleto fa5(a[5], b[5], carry[4], sum[5], carry[5]);
    Sumadorcompleto fa6(a[6], b[6], carry[5], sum[6], carry[6]);
    Sumadorcompleto fa7(a[7], b[7], carry[6], sum[7], carry[7]);
    assign Fcarry=carry[6];
endmodule

```

Figura 13, Código del Sumador de 8 bits.

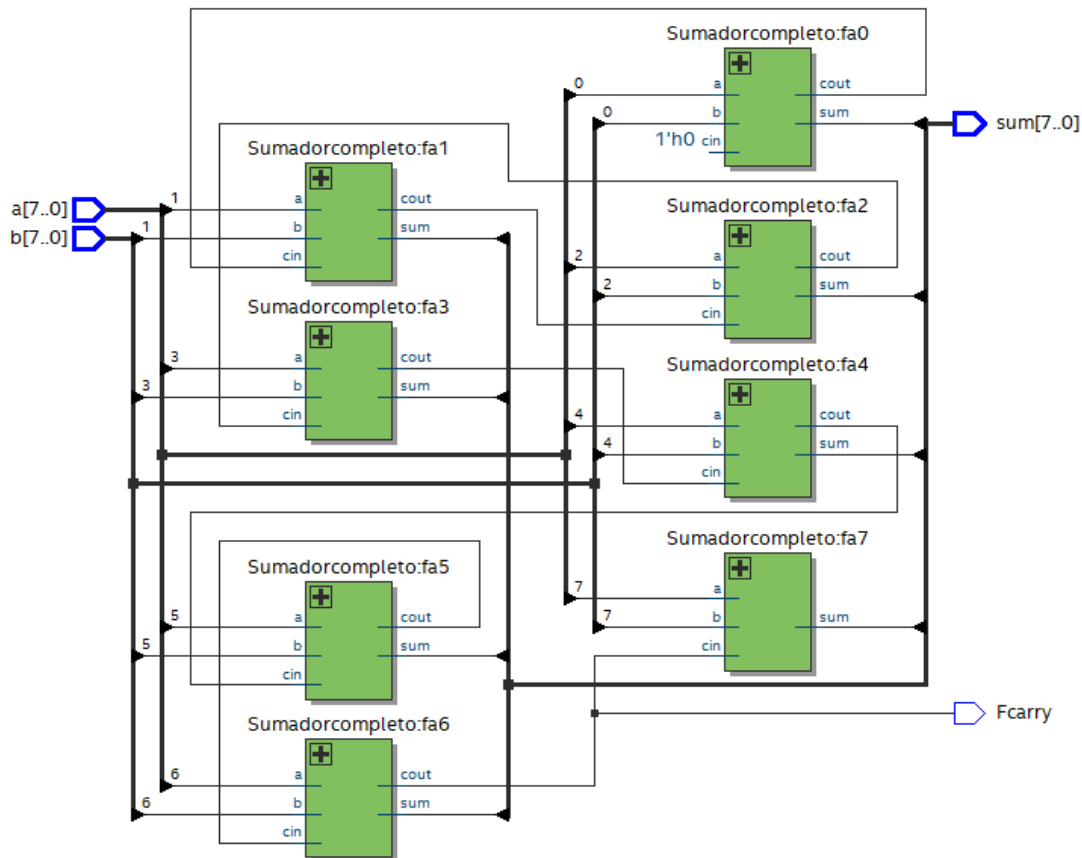


Figura 14, Compuertas Lógicas del sumador de 8 bits.

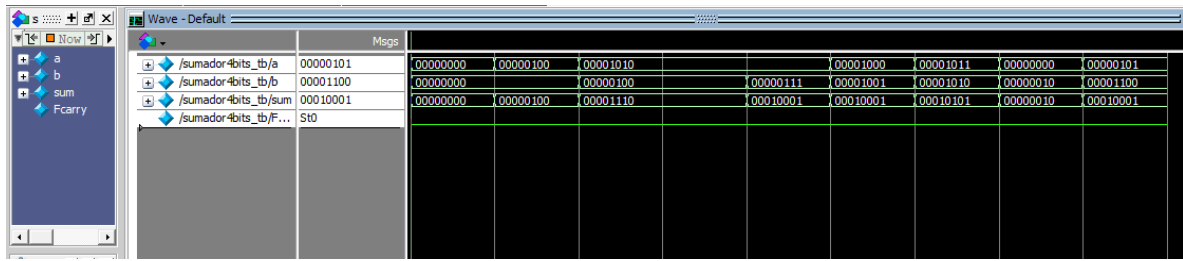


Figura 14, Simulación.